



Programación con JavaScript I



Actividad de Bienestar





Competencia

Crear archivos JavaScript que permiten
general **dinamismo básico** en sitio web



Requisitos técnicos

El aprendedor deberá tener **nociones básicas** de páginas web del lenguaje **HTML** y **CSS**. También debe **saber** utilizar un editor de texto y navegador web y saber instalar programas y ejecutarlos.



	Temas
1	Fundamentos de JavaScripts
2	Controles de flujo
3	Funciones
4	Objetos y arreglos
5	Programación orientada a objetos
6	Programación funcional
7	DOM
8	Manipulación del DOM



Un poco de MI

- Instructora del módulo JavaScript I y JavaScript II
- Experiencia en desarrollo y formación técnica
- Enfoque práctico y estructurado
- Me interesa que comprendan las bases, no solo que ejecuten código
- Acompaño el proceso paso a paso



¿Qué pueden esperar del curso?

- Nivel del curso: **fundamentos, no avanzado.**
- Enfoque: **práctico y aplicado**
- Ritmo: progresivo, se construye tema sobre tema.



¿Qué espero yo de ustedes?

- Revisar el temario disponible en la plataforma antes de cada sesión
- Realizar el prework correspondiente previo a la clase
- Asistencia y puntualidad
- Disposición para practicar durante la sesión
- Compromiso con su propio proceso de aprendizaje
- Tolerancia al error (el error es parte del proceso)



¿Cómo vamos a trabajar?

- Explicación breve de conceptos claves
- Práctica para afianzar conceptos
- Espacios definidos para resolver dudas del proyecto final
- Revisión de avances semanales del proyecto final



Planeación de avance proyecto

Semana	Avance	Orden	Tema
1	Avance 1	1	Fundamentos de JavaScripts
		2	Controles de flujo
2	Avance 2	3	Funciones
		4	Objetos y arreglos
3	Avance 3	5	Programación orientada a objetos
		6	Programación funcional
4	Avance 4	7	DOM
		8	Manipulación del DOM



Fundamentos de JavaScript

Programación con JavaScript I



Actividad Guiada

Parte 1

Duración: 75 minutos.



Pework 1: Preparación del entorno

1. Descarga e instalación de Git
2. Crea una cuenta en GitHub
3. Organización de tus proyectos
4. Entorno de desarrollo
5. Navegador web recomendado



Ejercicio 1

Crear tu primer proyecto con HTML y JavaScript, mostrando alertas y mensajes en la consola.



Parte 1: Configuración inicial

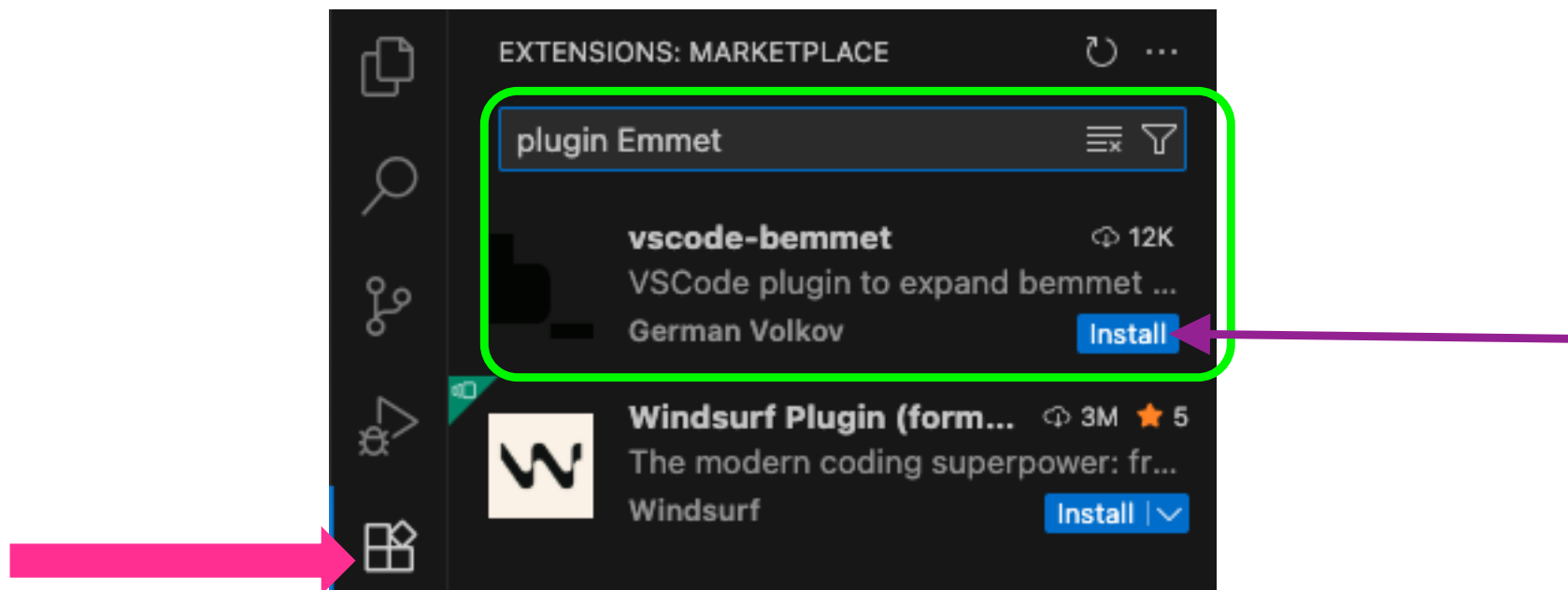
1. Abre la **carpeta de ejercicios** y crea un archivo `index.html`.
2. Carga el proyecto en **Visual Studio Code (VSC)**.
3. Crea la **estructura básica de HTML**:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
</body>
</html>
```



Parte 1: Configuración inicial

4. Instala el **plugin Emmet** en VSC para usar atajos (! crea la estructura HTML básica).





El **plugin Emmet** es una herramienta muy útil para desarrolladores web que **acelera la escritura de código HTML y CSS** mediante abreviaturas. Funciona como un “atajo inteligente” dentro de editores de código como **VS Code, Sublime Text, Atom, Brackets**, entre otros.



Emmet te permite **escribir menos y generar más**. Por ejemplo, en lugar de escribir todo el código HTML de un formulario, puedes usar una abreviatura que se expande automáticamente en código completo.

Abreviatura:

```
ul>li*5
```

Expansión al presionar **Tab**:

```
<ul>  
  <li></li>  
  <li></li>  
  <li></li>  
  <li></li>  
  <li></li>  
</ul>
```



Parte 1: Configuración inicial

5. Modifica el título:

```
<title>Hola Mundo con JS – [Tu nombre]</title>
```

6. Dentro del <body>, agrega:

```
<h1>Curso JavaScript</h1>  
<p>Nuestro primer script en JS</p>
```

7. Visualiza tu página arrastrando el archivo al navegador o usando **Live Server** en VSC.



Parte 2: Primeras alertas

1. Dentro del `<head>` o al final del `<body>`, agrega la etiqueta `<script>`:

```
<script>  
alert("Hola mundo con JS");  
alert("Bienvenido al curso");  
</script>
```

2. Para organizar mejor, crea un archivo externo **script.js**.
3. Mueve las alertas al archivo externo y enlázalo en tu HTML:

```
<script src="script.js"></script>
```

```
alert("Hola mundo con JS")  
alert("Bienvenido al curso")
```



Parte 2: Primeras alertas

4. Agrega dentro del HTML un mensaje:

```
<p>Hola mundo desde archivo externo</p>
```



Parte 3: Consola y comentarios

1. Imprime un mensaje en la consola:

```
console.log("muestra esto en la consola");
```

2. Envía a la consola el resultado de una operación aritmética:

```
console.log(5 + 3); // suma de 5 + 3
```

3. Agrega **comentarios** en tu **archivo script.js**:

- **Comentarios en bloque** para alertas
- **Comentarios individuales** para `console.log`

```
/* Esto  
   un ejemplo bloque  
*/  
  
// Ejem coment ind ...
```



Aprendizajes del Ejercicio 1

1. Estructura básica de un proyecto web
2. Uso de HTML y JavaScript
3. Primera interacción con JavaScript
4. Buenas prácticas iniciales
5. Herramientas de desarrollo



Ejercicio 2

Organización de archivos y uso de variables



Parte 1: Organización del proyecto

1. Renombra el archivo **script.js** a **hola-mundo.js**
2. Crea una carpeta llamada **js**.
3. Mueve ahí tu archivo **hola-mundo.js**.
4. Crea un nuevo archivo llamado **variables.js** dentro de la carpeta **js**.
5. Enlaza este archivo en **index.html** con la etiqueta:



```
<script src="js/variables.js"></script>
```



Parte 2: Declaración de variables básicas

```
let pais = "México";  
console.log(pais);
```

✓ Revisa el resultado en la **consola del navegador**.

Más variables

```
let continente = "América";  
console.log(pais, continente);
```

👉 Verás el resultado:

México América



Parte 3: Concatenación de variables

```
let antiguedad = 2019;  
let pais_y_continente = pais + " " + continente;  
  
alert(pais_y_continente);
```

👉 Resultado esperado:

```
México América
```



Parte 4: Actualización de valores

```
pais = "España";  
continente = "Europa";
```

👉 Resultado esperado:

España Europa

Cerrar

alert

📄 España – "Europa"

consola



Aprendizajes del Ejercicio 2

- Organización de archivos con carpetas.
- Declaración y actualización de variables en JavaScript.
- Uso de **console.log()** y **alert()** para mostrar resultados.
- Concatenación de variables para unir textos.



Receso

Duración: 10 minutos.

Regresamos: pm



Actividad Guiada

Parte 2



Ejercicio 3

Analizar el comportamiento de las variables declaradas con **var** y **let** en JavaScript, **identificar** cómo cambia su alcance según el contexto en el que se usan y **comprender** por qué resulta más seguro emplear **let** o **const** para escribir un código claro y predecible.



Parte 1: Configuración inicial

1. Crea un nuevo archivo js y nómbralo let-y-var.

```
▼ js  
JS let-y-var.js
```

2. Apunta el html para que tome la funcionalidad del nuevo archivo js.

```
<!-- Script de Jet y Var -->  
<script src="js/let-y-var.js"></script>
```



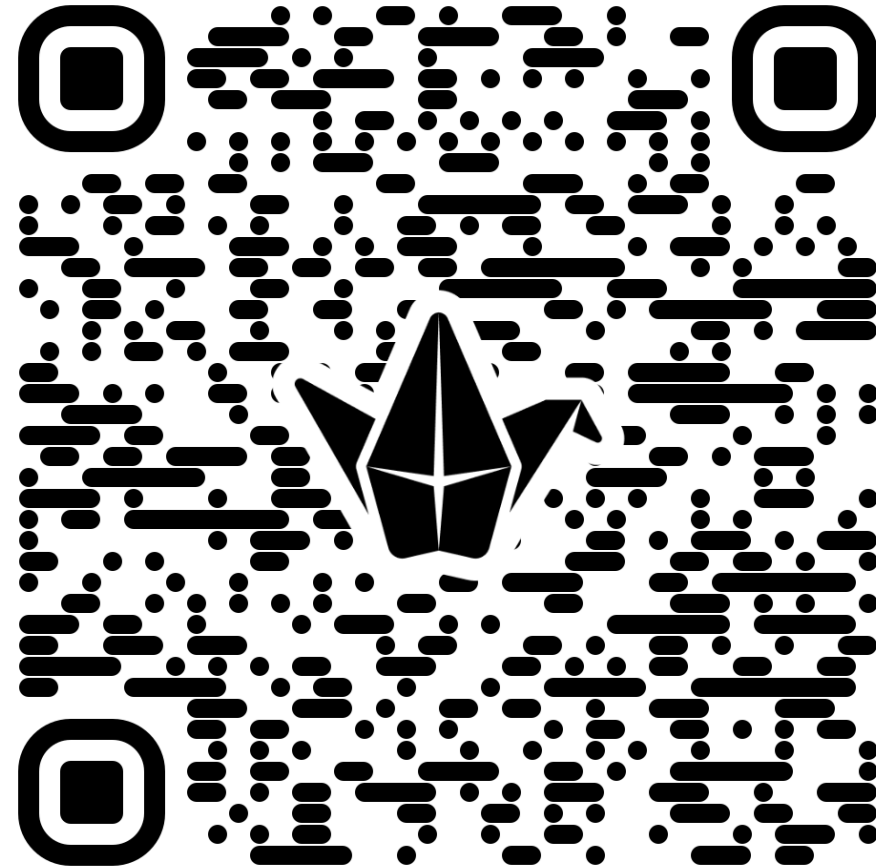
Parte 2: Agrega código al archivo let-y-var.

```
//prueba con var
var numero = 30;
console.log(numero); //valor esperado 30
if (true) {
    var numero = 40;
    console.log(numero); //valor esperado 40
}
console.log(numero); //¿Cuál es el valor de la variable numero?
//Prueba con let
var texto = "Curso de JS";
console.log(texto); //valor esperado "Curso de JS"
if (true) {
    let texto = "Curso de PHP";
    console.log(texto); // valor "Curso de PHP";
}
console.log(texto); // ¿Cuál es el valor de texto?
```



Parte 3: Análisis de código.

1. ¿Cuál es el valor de la variable número?
2. ¿Cuál es el valor de la variable texto?.





Aprendizajes del ejercicio 3

- `var` no respeta el alcance de bloque → puede sobrescribir variables.
- `let` respeta el bloque → mantiene variables independientes en cada contexto.
- Es recomendable usar **let** o **const** en lugar de `var` para evitar errores.
- La **prueba en consola** ayuda a verificar el comportamiento real del código.



Ejercicio 4

Comprender la diferencia entre variables y constantes en JavaScript, **aprender** a declarar y modificar variables con `let` y a declarar constantes con `const`, y **observar** el comportamiento del código al intentar cambiar los valores de una constante, utilizando la consola del navegador y la página web para visualizar los resultados.



Parte 1: Configuración inicial

1. Crea un nuevo archivo js y nómbralo **constantes**.

```
✓ js  
  JS constantes.js
```

2. Apunta el html para que tome la funcionalidad del nuevo archivo js.

```
<!-- Enlace al archivo JavaScript externo -->  
<script src="js/constantes.js"></script>
```



Parte 2: Variable web google

```
// 1. Crear una variable web con un valor inicial
let web = "https://www.google.com";

// 2. Crear una constante ip con un valor fijo
const ip = "192.168.1.100";

// Mostrar resultados iniciales
console.log("Valor inicial de web:", web);
console.log("Valor inicial de ip:", ip);
```



Parte 3: Actualiza variable web facebook.

```
// 3. Cambiar el valor de la variable web
web = "https://www.facebook.com";
console.log("Nuevo valor de web:", web);

// 4. Intentar cambiar el valor de la constante ip
try {
  ip = "10.10.1.100"; // Esto generará un error
} catch (error) {
  console.error("Error al intentar cambiar ip:", error.message);
}

// 5. Mostrar mensaje en pantalla
document.write("<h2>Ejercicio 4 en ejecución</h2>");
document.write("<p>Revisa la consola para observar el comportamiento de la variable y la constante.</p>");
```




Try - catch

- 'try' sirve para ejecutar código que podría causar un error.
- 'catch' permite capturar ese error y manejarlo sin que el programa se detenga.

```
try {  
  ip = "10.10.1.100"; // Esto generará un error  
} catch (error) {  
  console.error("Error al intentar cambiar ip:", error.message);  
}
```

En este caso, se captura el intento de modificar una constante y se muestra un mensaje claro en la consola.



Aprendizajes del ejercicio 4

1. Declaración de variables y constantes
2. Modificación de valores
3. Uso de la consola del navegador
4. Manejo de errores con try-catch
5. Buenas prácticas



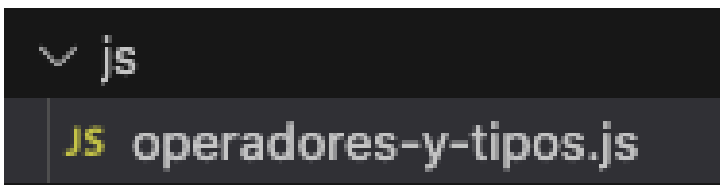
Ejercicio 5

Explorar el uso de operadores aritméticos y la manipulación de diferentes tipos de datos en JavaScript, **aprender** a realizar operaciones entre números y cadenas de texto, **comprender** cómo se comporta la concatenación frente a la suma aritmética, y **practicar** la conversión de tipos de datos y la verificación de tipos mediante la consola del navegador.



Parte 1: Configuración inicial

1. Crea un nuevo archivo js y nómbralo **operadores-y-tipos.js**.



2. Apunta el html para que tome la funcionalidad del nuevo archivo js.

```
<!-- Enlace al archivo JavaScript externo -->  
<script src="js/operadores-y-tipos.js"></script>
```



Parte 2: Creación de variables

1. Crea las siguientes variables **numero1=7**, **numero2=15**.

```
let numero1 = 7;  
let numero2 = 15;
```

2. Ahora, crea las variables: **suma**, **resta**, **division**, **multiplicacion** y realiza las operaciones aritméticas sobre las variables **numero1** y **numero2**.

```
let suma = numero1 + numero2;  
let resta = numero1 - numero2;  
let multiplicacion = numero1 * numero2;  
let division = numero1 / numero2;
```



Parte 3: Mostrar resultados

1. Muestra en una alerta el resultado de las operaciones precedido por el texto “el resultado de la [operación] es: ”.

```
alert("El resultado de la suma es: " + suma);  
alert("El resultado de la resta es: " + resta);  
alert("El resultado de la multiplicación es: " + multiplicacion);  
alert("El resultado de la división es: " + division);
```

2. La pantalla debe mostrar los resultados así





Parte 4: Declarar variables de diferentes tipos

1. Declara las variables `numero_entero=24`, `cadena_texto = "Hola 'que' tal"`, `verdadero_o_falso=true`, `numero_falso="33"`.

```
let numero_entero = 24;  
let cadena_texto = "Hola 'que' tal";  
let verdadero_o_falso = true;  
let numero_falso = "33";
```

2. Suma la variable `numero_entero` y `numero_falso`. ¿Cuál es el resultado y por qué?

```
let suma_incorrecta = numero_entero + numero_falso;
```



Parte 5: Conversión de numero_falso

1. Convierte el contenido de la variable numero_falso en un entero. ¿Cuál fue el resultado y por qué?

```
let numero_falso_entero = parseInt(numero_falso);  
// Ahora la suma será aritmética  
let suma_correcta = numero_entero + numero_falso_entero;  
alert("Convertido a entero: numero_entero + numero_falso_entero = " + suma_correcta);
```

2. Convierte la variable numero_entero a string y súmalo al número 9. ¿El resultado fue un número 33? ¿Por qué?

```
let numero_entero_string = numero_entero.toString();  
let suma_string = numero_entero_string + 9;  
alert("Convertido a string y sumado a 9: " + suma_string);
```




Parte 6: Impresión en consola

1. Imprime en la consola el tipo de dato de las variables **numero_entero**, **cadena_texto**, **verdadero_o_falso**, **numero_falso**.

```
console.log("Tipo de numero_entero:", typeof numero_entero); // number
console.log("Tipo de cadena_texto:", typeof cadena_texto); // string
console.log("Tipo de verdadero_o_falso:", typeof verdadero_o_falso); // boolean
console.log("Tipo de numero_falso:", typeof numero_falso); // string
console.log("Tipo de numero_falso_entero:", typeof numero_falso_entero); // number
```



Aprendizajes del ejercicio 5

1. Operaciones aritméticas básicas
2. Alertas para mostrar resultados
3. Diferentes tipos de datos
4. Conversión de tipos de datos
5. Uso de la consola
6. Comprender el comportamiento de JavaScript



Cierre

Tema 1: Fundamentos de JavaScript



- Crear proyectos básicos con **HTML y archivos JS externos**.
- Mostrar información en **alertas** y en la **consola del navegador**.
- Declarar **variables y constantes**, comprendiendo sus diferencias y limitaciones.
- Realizar **operaciones aritméticas** y entender cómo JavaScript maneja la **concatenación y los tipos de datos**.
- Convertir y manipular **tipos de datos**, utilizando funciones como `parseInt()` y `toString()`.
- Capturar errores con **try-catch**, evitando que los errores detengan la ejecución del programa.



Estos conceptos son la **base fundamental** para desarrollar aplicaciones interactivas y dinámicas en JavaScript. La práctica constante y la experimentación con variables, operadores y tipos de datos te permitirán ganar confianza y seguridad para avanzar a temas más complejos.